

Answer-File Generator Options: `UnattendedWinstall` and `unattend-generator`

Like `docs/rufus-feature-options.md`, this is an **options inventory from a reference tool consulted for technique**, not a builder-vs-builder comparison (that's `docs/script-comparison.md`). `UnattendedWinstall` (`reference/UnattendedWinstall/`) and `unattend-generator` (`reference/unattend-generator/`, source for the [Windows Unattended Answer File Generator](#)) are grouped together here because they're the same category of tool: both produce a hand-crafted/generated `autounattend.xml` that does *all* of its customization declaratively or via scripts embedded in the answer file — neither one touches `install.wim` / `boot.wim` offline the way this repo, `ntdevlabs`, and `tiny11-automated` do. No integration exists yet (see `CLAUDE.md`); this is reference-only.

The core difference from this repo: *when the work happens*

This repo (and its `ntdevlabs` / `tiny11-automated` ancestry) does essentially all its customization **offline, before the ISO is even built** — DISM mounts `install.wim`, the pipeline deletes/edits things inside it, DISM commits and unmounts. By the time Setup runs, the work is already done; the answer file's job is just to skip prompts (product key, EULA, account creation).

`UnattendedWinstall` and `unattend-generator` instead do most of their work **live, during Windows Setup**, by embedding PowerShell into the answer file at specific passes:

- `reference/unattend-generator/modifier/Bloatware.cs` — for each selector a user picked (`install.wim` stays untouched), it writes a block like:

```
Get-AppxProvisionedPackage -Online | Where-Object { $_.DisplayName -eq '<selector>' } |
Remove-AppxProvisionedPackage -AllUsers -Online -ErrorAction Continue
```

into a generated `RemovePackage.ps1` (same pattern for `RemoveCapability.ps1` via `Remove-WindowsCapability` and a Feature variant via `Disable-WindowsOptionalFeature`), invoked from the **specialize** pass (`SpecializeScript.InvokeFile`, see `modifier/Script.cs` / `modifier/Specialize.cs`).

- `reference/UnattendedWinstall/autounattend.xml` does the same thing by hand: a inline `<SynchronousCommand>` block in the specialize pass running `Get-AppxProvisionedPackage -Online | Remove-AppxProvisionedPackage` for a curated list.
- `reference/unattend-generator/modifier/DefaultUser.cs` is the same idea applied to the Default user profile: it `reg load s HKU\DefaultUser` from `C:\Users\Default\NTUSER.DAT` *during the specialize pass*, runs a PowerShell script against it, then unloads — functionally the same registry-hive edit this repo's `Mount-OfflineRegistryHives` / `Dismount-OfflineRegistryHives` does, just at a different time (live during Setup vs. offline before Setup ever starts).
- `reference/unattend-generator/modifier/FirstLogon.cs` embeds a script invoked from the **oobeSystem** pass instead — i.e. after the user's own account exists, equivalent to this repo's

`Install-VirtioGuestToolsAtFirstLogon` first-logon-at-OOBE pattern, just generalized to any tweak instead of only guest-tool install.

This is a genuine third timing option worth naming explicitly (see `docs/build-options-reference.md`'s taxonomy): **offline pre-boot** (this repo's default) vs. **Setup-time embedded script** (specialize or oobeSystem pass — these two tools' primary technique) vs. **declarative answer-file setting** (a `<Component>` value Setup itself interprets, no script at all — e.g. this repo's `Add-AnswerFileLocale`, `Add-AnswerFileLocalAccount`) vs. **post-OOBE first-logon** (needs a real logged-on user, e.g. scheduled task or Run-key script).

Trade-offs of the Setup-time-embedded-script approach vs. this repo's offline approach:

- *Pro:* no DISM mount/unmount cycle at build time; the generated ISO is just a stock Windows ISO plus one `autounattend.xml` — much faster to produce, and the removal logic runs with full context of whatever the live install actually provisioned (no risk of operating on a stale snapshot of the image).
- *Con:* the removal work happens on every install run, adding to install time, and if a step fails Setup's error handling is far blunter than this repo's try/catch-and-continue DISM calls — `Get-AppxProvisionedPackage` failures are swallowed with `-ErrorAction Continue` precisely because there's no good recovery path mid-Setup.

Security-relevant finding: sensitive file cleanup (now fixed)

`reference/unattend-generator/modifier/Delete.cs` (`DeleteModifier`) deletes `C:\Windows\Panther\unattend.xml`, `unattend-original.xml`, and `C:\Windows\Setup\Scripts\Wifi.xml` from the **installed system** at first logon (via `FirstLogonScript.Append`), specifically because Setup copies the answer file — Wi-Fi passwords, any embedded account password, product keys and all — into `Panther\` on the final installed drive, readable by anyone who later gets a shell on the machine.

This repo now does the equivalent. `-LocalAccountName` (`CLAUDE.md`'s own documented caveat) sets the account's password to the plaintext value of the account name — "fine for disposable/dev/VM images... never use on an image reachable by an untrusted network or user" — and that plaintext, plus a real `-ProductKey` if given, used to sit unremoved in `C:\Windows\Panther\unattend.xml` and this repo's own `Windows\System32\Sysprep\autounattend.xml` copy (written by `Enable-LocalAccountOOBE`) on every installed machine. `Remove-SensitiveAnswerFilesAtFirstLogon` (`asl-win11.functions.ps1`) now stages a `RunOnce` entry that deletes all three known copies (`Sysprep\autounattend.xml`, `Panther\unattend.xml`, `Panther\unattend-original.xml`) at first logon, same technique as `DeleteModifier`, applied unconditionally (it's a no-op `del` on whichever paths don't exist for a given build, e.g. the Sysprep copy under `-KeepCorporateApps`). This covers the three paths this repo's own pipeline and Setup's documented behavior are known to populate; it hasn't been verified against every other file Setup's logging may incidentally leave under `Windows\Panther\` on a given Windows 11 build, so treat it as closing the known gap, not as an exhaustive guarantee.

UnattendedWinsta11 specifically

A single curated, hand-maintained autounattend.xml (not a code generator — the generator is actually a separate, unrelated project by the same author, Winhance) plus extensive inline XML comments. Per its README: bypasses the Windows 11 hardware checks, skips the forced Microsoft-account OOBE screen, removes all preinstalled bloatware apps except Notepad/Calculator/Paint/Snipping Tool (Copilot, OneDrive, and Edge included), disables Recall, and applies a bundle of "Optimizations" — privacy/telemetry, a power-plan import, and gaming/performance service+scheduled-task tweaks. All overlap with tweaks already in docs/tweak-catalog.md ; nothing here is a new tweak this repo lacks, just the same tweaks applied at Setup-time instead of offline.

unattend-generator specifically: broader surface than bloatware removal

Beyond Bloatware.cs / Bypass.cs / Delete.cs / DefaultUser.cs / FirstLogon.cs above, its modifier/ directory covers categories this repo doesn't touch at all — listed here as candidates for a future look, not implemented:

File	What it configures
Optimizations.cs	Visual effects presets (best-performance/best-appearance), Sticky Keys behavior, Caps/Num/Scroll Lock initial state and toggle behavior, Start menu pins/tiles, taskbar icon set, process-creation auditing
AppLocker.cs	AppLocker application whitelisting policy
Lockout.cs / PasswordExpiration.cs	Account lockout policy, password expiration policy
Wifi.cs	Embeds a Wi-Fi profile (SSID + key) so Setup joins a network unattended
Components.cs	Toggles optional Windows components/features declaratively
Accessibility.cs	Accessibility (narrator/magnifier/etc.) defaults
ComputerName.cs	Sets the computer name from the answer file
ProductKey.cs	Same category as this repo's -ProductKey /GVLK handling, implemented as its own modifier

None of these map to a current asl-win11maker.ps1 parameter. They're genuinely out of this project's current scope (bloat/telemetry removal + hardware compat + driver injection), but worth knowing they exist if a future ask is "can we also configure X during the build."

See also

- docs/tweak-catalog.md — the per-tweak source-of-truth table these two tools' bloatware/telemetry coverage maps onto.
- docs/rufus-feature-options.md — the equivalent write-up for Rufus, the other non-WIM-servicing reference tool.

- `docs/build-options-reference.md` — where the offline/Setup-time/first-logon phase distinction from this doc is applied across every actual `asl-win11maker.ps1` option.